

腾讯混元AIGC前端架构师

一面（技术基础） 1-6

1. 请简述 JavaScript 的事件循环机制，并说明在处理 AI 流式响应时，宏任务与微任务的影响。
2. 在前端处理大模型返回的 Stream 数据时，你会用到哪些 Web API?
3. 如何实现一个高性能的 Markdown 渲染组件，支持实时解析 AI 输出?
4. 请手写一个简单的 Promise.allSettled 的实现。
5. CSS 如何实现一个类似 ChatGPT 的光标闪烁效果?
6. 谈谈你对 TypeScript 中泛型和 infer 关键字的理解，并举一个在 AI 参数配置中的应用场景。

二面（深入原理与项目） · 1-8

1. AI 对话界面中，当消息列表非常长时，如何保证滚动和渲染的性能?
2. 实现 AI 打字机效果时，如何解决“字符吞噬”或“渲染卡顿”问题?
3. 在前端如何实现 Prompt 的版本管理和动态替换变量?
4. 谈谈 RAG（检索增强生成）在前端侧可以做哪些优化?
5. 如何设计一个通用的 AI Chat 组件库? 需要考虑哪些扩展性?
6. Web Worker 在大模型前端应用中有哪些使用场景?
7. 你们是如何处理 AI 接口超时和异常重试策略的?
8. 谈谈前端如何防御 Prompt 注入攻击?

三面（架构与综合能力） · 1-6

1. 如果让你从零搭建腾讯混元的前端架构，你会如何设计以支撑多业务接入?
2. 在 AIGC 时代，你认为前端工程师的核心竞争力发生了什么变化?
3. 如何评估一个 AI 前端应用的用户体验? 有哪些关键指标?
4. 聊聊你在项目中遇到的最难的技术挑战，你是如何解决的?
5. 对于 WebGPU 和 WebAssembly 在前端 AI 领域的应用，你有什么看法?
6. 如果团队需要引入 AI 辅助编程（如 Copilot），你会如何制定落地标准?

一面（技术基础）

1. 请简述 JavaScript 的事件循环机制，并说明在处理 AI 流式响应时，宏任务与微任务的影响。

难度：★★

考点：Event Loop、异步调度

答案要点：

事件循环是 JS 处理异步操作的核心机制，通过执行栈、微任务队列和宏任务队列的协调运行。

- 执行顺序：同步代码 -> 微任务（Promise.then, MutationObserver） -> 渲染更新 -> 宏任务（setTimeout, I/O）。
- 流式响应影响：Fetch API 的流式读取通常在微任务中处理，如果微任务过重会阻塞 UI 渲染。
- 渲染时机：浏览器通常在宏任务执行前尝试渲染，频繁的微任务更新可能导致打字机效果不流畅。
- 任务拆解：对于复杂的 AI 文本解析，应考虑通过 requestAnimationFrame 或宏任务拆分长任务。

常见坑：

- 认为 Promise 构造函数内部是异步的。
- 混淆 Node.js 环境与浏览器环境的事件循环差异。

追问：

- 如果在微任务里递归创建微任务，会导致什么后果？

2. 在前端处理大模型返回的 Stream 数据时，你会用到哪些 Web API？

难度：★★

考点：Fetch API、ReadableStream、TextDecoder

答案要点：

主要使用 Fetch API 配合 ReadableStream 接口，通过 TextDecoder 将二进制流转为字符串。

- ReadableStream：通过 response.body.getReader() 获取只读流读取器。
- TextDecoder：处理 UTF-8 编码，支持处理被截断的字节序列（stream: true 模式）。
- 循环读取：使用 while(true) 配合 reader.read() 递归获取 value 和 done 状态。
- 异常处理：需要处理网络中断、读取超时以及 reader.cancel() 的手动关闭。

常见坑：

- 忘记处理 TextDecoder 在字符截断时的拼包问题。
- 未对 reader 进行错误捕获导致内存泄漏。

追问：

- SSE（Server-Sent Events）和 Fetch Stream 相比有什么优缺点？

3. 如何实现一个高性能的 Markdown 渲染组件，支持实时解析 AI 输出？

难度：★★

考点：正则解析、AST、DOM 性能

答案要点：

高性能渲染需要结合增量解析和虚拟 DOM 优化，避免全量重新渲染。

- 增量解析：记录已解析的索引，仅对新到达的 Stream 片段进行解析和追加。
- 库选型：常用 markdown-it 或 marked，配合 highlight.js 或 prism.js 处理代码高亮。
- 渲染优化：在 React/Vue 中使用 memo 缓存已生成的节点，防止光标跳动。
- 安全性：必须使用 DOMPurify 等工具过滤 XSS 攻击向量。

常见坑：

- 每次流更新都重新渲染整个 Markdown 文档，导致 CPU 占用过高。
- 代码高亮插件在流式输出时频繁重绘导致闪烁。

追问：

- 如何在 Markdown 渲染过程中支持自定义的 AI 交互组件（如：图表、按钮）？

4. 请手写一个简单的 Promise.allSettled 的实现。

难度：★★

考点：Promise 原理、手写代码

答案要点：

Promise.allSettled 会等待所有 Promise 执行完毕，无论成功或失败，并返回包含状态和值的对象数组。

- 计数器：记录已完成（fulfilled 或 rejected）的 Promise 数量。
- 结果存储：按输入顺序存储每个 Promise 的执行结果对象。
- 状态封装：成功返回 `{ status: 'fulfilled', value }`，失败返回 `{ status: 'rejected', reason }`。
- 边界处理：处理空数组的情况。

代码块

```
1  function allSettled(promises) {
2    return new Promise((resolve) => {
3      const results = [];
4      let count = 0;
5      if (promises.length === 0) resolve([]);
6      promises.forEach((p, i) => {
7        Promise.resolve(p).then(
8          value => {
9            results[i] = { status: 'fulfilled', value };
10           if (++count === promises.length) resolve(results);
11         },
12         reason => {
13           results[i] = { status: 'rejected', reason };
14           if (++count === promises.length) resolve(results);
15         }
16       });
17     });
18   }
```

```
17     });  
18     });  
19 }
```

常见坑：

- 结果数组顺序与输入顺序不一致。
- 未使用 `Promise.resolve` 包装非 `Promise` 类型的值。

5. CSS 如何实现一个类似 ChatGPT 的光标闪烁效果？

难度：★

考点：CSS 动画、伪元素

答案要点：

利用伪元素和 `keyframes` 动画模拟垂直线条的显隐。

- 伪元素：在内容容器后添加 `::after` 伪元素。
- 样式模拟：设置 `width`, `height`, `background-color` 为黑色或主题色。
- 动画：使用 `opacity` 从 0 到 1 的循环切换，或 `visibility` 切换。
- 步进函数：使用 `steps(1)` 或 `linear` 达到跳动感。

常见坑：

- 动画导致页面频繁重排（`Reflow`），应优先使用 `opacity`。
- 光标在换行时的位置偏移问题。

6. 谈谈你对 TypeScript 中泛型和 `infer` 关键字的理解，并举一个在 AI 参数配置中的应用场景。

难度：★★★

考点：TS 高级类型

答案要点：

泛型提供类型变量化能力，`infer` 用于在模式匹配中提取类型。

- 泛型：实现组件或函数对多种数据类型的兼容，保持类型收敛。
- `infer`：在条件类型中声明一个变量来承载提取出的类型。
- 应用场景：自动推断 AI 模型接口的返回类型。例如从一个复杂的 `ModelConfig` 中提取出特定的 `Response` 类型。

代码块

```
1 type GetResponse<T> = T extends { predict: (...args: any[]) => Promise<infer  
  R> } ? R : never;
```

常见坑：

- infer 只能在 extends 的 true 分支中使用。
- 泛型约束过于宽松导致失去类型检查意义。

二面（深入原理与项目）

1. AI 对话界面中，当消息列表非常长时，如何保证滚动和渲染的性能？

难度：★★★

考点：虚拟列表、渲染优化、内存管理

答案要点：

长列表优化核心在于减少 DOM 节点数和降低渲染频率。

- 虚拟滚动：只渲染可视区域内的消息节点，通过 offset 模拟滚动高度。
- 动态高度计算：AI 消息高度不固定，需实时测量或使用 ResizeObserver 监听高度变化并更新缓存。
- 渲染分片：流式输出时，采用缓冲区机制，每隔 100-200ms 批量更新一次 UI。
- 内存清理：对过期的上下文或不可见的大图、附件进行资源释放。

常见坑：

- 滚动过快时出现白屏。
- 频繁触发 setState 导致 React Fiber 树反复调和。

追问：

- 虚拟列表在开启浏览器原生搜索（Ctrl+F）时失效怎么办？

2. 实现 AI 打字机效果时，如何解决“字符吞噬”或“渲染卡顿”问题？

难度：★★★

考点：流式调度、RequestAnimationFrame

答案要点：

打字机效果不能直接透传后端 Stream 片段，需要建立前端缓冲区进行平滑调度。

- 缓冲区队列：将后端推送的 chunk 放入队列，前端按固定频率（如 50ms/字）取出。
- 调度器：使用 requestAnimationFrame 驱动字符渲染，确保与屏幕刷新率同步。
- 动态调速：当缓冲区堆积过多时加速播放，堆积过少时减速，保证视觉连续性。
- 文本处理：处理多字节字符（如 Emoji）被截断导致的乱码问题。

常见坑：

- 直接使用 setInterval 导致在后台标签页时逻辑堆积。
- 忽略了 Markdown 语法标记在未闭合时的渲染逻辑。

3. 在前端如何实现 Prompt 的版本管理和动态替换变量？

难度：★★

考点：字符串模板、工程化

答案要点：

Prompt 管理需要实现模板化和结构化，通常采用 DSL 或简单的占位符。

- 模板引擎：使用类似 Mustache 的占位符 `{{variable}}` 进行正则替换。
- 结构化定义：将 Prompt 拆分为 System Role, Context, User Input 等模块。
- 版本控制：在前端配置中心（如 Apollo 或自建后台）存储 Prompt ID 和版本号。
- 动态注入：在请求拦截器中根据当前业务上下文自动注入用户信息、时间等变量。

常见坑：

- 直接在代码里硬编码长字符串，难以维护。
- 替换变量时未进行转义，导致 Prompt 注入攻击。

4. 谈谈 RAG（检索增强生成）在前端侧可以做哪些优化？

难度：★★★

考点：RAG、前端缓存、向量化

答案要点：

前端在 RAG 流程中主要承担数据预处理、本地缓存和结果展示的任务。

- 本地检索：对于小规模知识库，可使用 WebAssembly 版的向量数据库（如 DuckDB, Voy）在浏览器端检索。
- 预取策略：根据用户输入意图，提前触发相关文档的加载。
- 引用溯源：解析 AI 返回的引用标记，实现点击跳转到 PDF 特定页码或文档锚点。
- 结果过滤：在前端对检索到的噪声数据进行二次过滤，减少 Token 消耗。

常见坑：

- 前端处理过大向量索引导致页面崩溃。
- 引用链接失效或指向错误。

5. 如何设计一个通用的 AI Chat 组件库？需要考虑哪些扩展性？

难度：★★★

考点：组件设计模式、Headless UI

答案要点：

通用 AI 组件应遵循逻辑与 UI 分离的原则。

- 核心逻辑层（Headless）：封装流式请求、历史记录管理、状态机（思考中、输出中、错误）。
- 视图层：提供默认样式，同时支持 Slot/Render Props 供用户自定义消息气泡、输入框。

- 插件系统：支持扩展工具调用（Function Calling）的展示插件，如天气插件、图表插件。
- 适配器：支持接入不同的 LLM 后端协议（OpenAI, Anthropic 等）。

常见坑：

- 样式污染，未采用 CSS Modules 或 Tailwind。
- 逻辑耦合严重，难以从 React 迁移到 Vue。

6. Web Worker 在大模型前端应用中有哪些使用场景？

难度：★★

考点：多线程、Web Worker

答案要点：

Web Worker 用于将重计算任务移出主线程，防止 UI 掉帧。

- Tokenizer 计算：在前端计算输入文本的 Token 数量。
- 向量计算：执行余弦相似度计算等数学运算。
- 大规模数据解析：解析数 MB 的 JSON 响应或 Markdown AST 构建。
- 离线模型运行：使用 WebLLM 等框架在 Worker 中运行轻量化模型。

常见坑：

- 频繁的 PostMessage 导致序列化开销过大。
- 在 Worker 中尝试操作 DOM。

7. 你们是如何处理 AI 接口超时和异常重试策略的？

难度：★★

考点：网络容错、用户体验

答案要点：

针对 AI 接口响应慢的特性，需要多层次的容错机制。

- 指数退避重试：失败后按 1s, 2s, 4s 间隔重试，避免雪崩效应。
- 分段超时：设置首包超时（TTFT）和整体响应超时。
- 状态恢复：在重试时保留已输出的内容，实现断点续传（需后端支持）。
- 降级方案：当模型不可用时，自动切换到轻量级模型或展示友好提示。

常见坑：

- 自动重试导致用户 Token 消耗翻倍。
- 界面上没有明确的错误反馈，用户陷入无尽等待。

8. 谈谈前端如何防御 Prompt 注入攻击？

难度：★★★

考点：安全、Prompt Injection

答案要点：

前端是防御的第一道防线，通过输入校验和结构化封装降低风险。

- 输入清洗：过滤掉类似 "Ignore previous instructions" 的敏感关键词。
- 结构化输入：不直接拼接字符串，而是通过 ChatML 等格式化 API 发送请求。
- 长度限制：严格限制用户输入的字符长度，防止超长文本绕过检测。
- 内容安全策略（CSP）：防止 AI 输出的恶意脚本在页面执行。

常见坑：

- 认为前端过滤就足够了，忽略了后端校验的重要性。
 - 过滤规则过于死板，影响正常业务表达。
-

三面（架构与综合能力）

1. 如果让你从零搭建腾讯混元的前端架构，你会如何设计以支撑多业务接入？

难度：★★★★

考点：架构设计、中台化、微前端

答案要点：

需要构建一个“内核+插件”的平台化架构。

- 核心 SDK：封装鉴权、流式通信、中转网关、监控埋点等基础能力。
- 业务适配层：通过配置化方式适配不同业务的 UI 风格和功能开关。
- 插件化机制：定义标准的 AI 能力接口，各业务可独立开发并注入自定义的 Tool Call 渲染组件。
- 离线化与性能：利用 Service Worker 缓存静态资源，建立统一的端侧向量缓存池。
- 监控体系：建立专门的 AI 指标监控，如首字延迟、Token 吞吐量、用户点踩率等。

2. 在 AIGC 时代，你认为前端工程师的核心竞争力发生了什么变化？

难度：★★

考点：行业洞察、职业规划

答案要点：

前端工程师正从“界面实现者”转向“AI 能力集成者与交互设计师”。

- 交互范式：从确定性的 GUI 转向不确定性的 LUI（Language User Interface），需掌握意图识别和容错设计。
- 技术栈扩展：需要了解模型基本原理（如 Context Window, Temperature）、向量数据库、Prompt 工程。
- 工程化深度：不仅是打包编译，更包括模型评测自动化、数据反馈闭环的构建。
- 效率工具：熟练使用 AI 工具重构开发流程，提升代码产出质量。

3. 如何评估一个 AI 前端应用的用户体验？有哪些关键指标？

难度：★★★

考点：性能指标、UX 设计

答案要点：

除了传统的性能指标，更需关注 AI 特有的交互反馈。

- TTFT (Time to First Token)：首字响应时间，直接影响用户焦虑感。
- 文本生成速率：是否符合人类阅读速度，过快或过慢都不利于体验。
- 交互一致性：Markdown 渲染是否准确，代码块是否支持一键复制。
- 幻觉反馈机制：是否提供便捷的纠错、重试或人工介入入口。
- 资源消耗：端侧模型运行时的 CPU/显存占用情况。

4. 聊聊你在项目中遇到的最难的技术挑战，你是如何解决的？

难度：★★★

考点：问题解决能力、技术深度

答案要点：

(此题为开放性问题，需结合实际经验)

- 描述背景：业务场景是什么，遇到了什么瓶颈（如：长对话内存泄漏、复杂 Tool Call 渲染冲突）。
- 分析过程：使用了什么工具（Chrome DevTools, Memory Profile）定位根因。
- 方案对比：对比了哪几种方案，为什么选择了最终这一种。
- 最终结果：通过量化数据（如：FPS 提升、内存下降 %）证明价值。

5. 对于 WebGPU 和 WebAssembly 在前端 AI 领域的应用，你有什么看法？

难度：★★★

考点：前沿技术、端侧 AI

答案要点：

这是实现“端侧 AI”和“隐私计算”的关键技术。

- WebGPU：提供更直接的 GPU 访问能力，相比 WebGL 大幅提升了矩阵运算效率，是浏览器运行 Transformer 模型的基石。
- WebAssembly：用于移植 C++/Rust 编写的模型推理引擎（如 llama.cpp），实现接近原生的执行速度。
- 价值：减少服务端算力成本，降低响应延迟，保护用户隐私数据不离端。
- 现状：生态尚在早期，兼容性和模型压缩技术（量化）是目前落地的主要挑战。

6. 如果团队需要引入 AI 辅助编程（如 Copilot），你会如何制定落地标准？

难度：★★

考点：技术管理、工程效率

答案要点：

落地标准应围绕“安全、效率、质量”三个维度。

- 安全合规：制定代码脱敏规范，禁止上传核心密钥或敏感业务逻辑。
- 效率度量：通过代码采纳率、需求交付周期缩短比例进行量化评估。
- 质量把控：AI 生成的代码必须经过人工 Code Review 和单元测试覆盖，不能直接上线。
- 知识沉淀：建立团队内部的 Prompt 库，分享高频场景的提问技巧。